
FarmHub

FarmHub

Android Application for Agricultural Equipment Rental and Product Shopping

MINI-PROJECT REPORT

Submitted in partial fulfilment of the requirements for the

CSA- 5005 - Object Oriented Technology Lab

Submitted by

Jagruti Gadekar - 2621

Magnus Rodrigues - 2628

Dhanashri Ghumate - 2617

Department of Masters Of Computer Application

[Goa University]

Academic Year: 2026–2027

DECLARATION

We hereby declare that the mini-project entitled “FarmHub: Android Application for Agricultural Equipment Rental and Product Shopping” has been developed by our group of three members as part of the academic requirements for Object Oriented Technology Lab at Goa Business School during the academic year 2026–2027.

This project has been carried out under the guidance of Mr Hanumant Harichandra Redkar. The report presents the objectives, design, technologies, implementation, and testing of the proposed application.

We further declare that the information included in this report accurately represents the work carried out by our group, and that the sources referred to during the preparation of this report are acknowledged.

Group Members

1. Jagruti Tulsidas Gadekar
2. Magnus Rodrigues
3. Dhanashri Ghumate

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide,Mr Hanumant Harichandra Redkar, for providing guidance, suggestions, and support throughout the development of FarmHub. The guidance received during the planning, implementation, and review stages helped us understand the importance of translating a project idea into a functional software application.

We are thankful to the Head of the Department and the faculty members of Master of Computer Application, Goa University, for providing us with the opportunity to undertake this project and for encouraging us to apply the concepts learned during our academic studies.

We also acknowledge the resources and technologies that supported the development of FarmHub. Android development tools helped us create the application interface and implement the required functionality. Java provided the programming foundation, while Firebase Authentication and Cloud Firestore supported user authentication and cloud-based data operations.

We would also like to thank our teammates for their cooperation during requirement discussions, coding, module integration, troubleshooting, testing, and documentation. Dividing the work into the agricultural product marketplace, equipment rental and booking, and authentication and integration modules helped us organise the development process.

Finally, we express our gratitude to our families and everyone who encouraged us during the completion of this project. Their support motivated us to continue working through development challenges and prepare the project for academic evaluation.

ABSTRACT

FarmHub is an Android-based application developed to simplify agricultural equipment rental and agricultural product shopping. Farmers often face difficulties in purchasing expensive agricultural machinery and finding suitable equipment for temporary use. They may also need to visit different shops or contact multiple sellers to obtain agricultural products. FarmHub aims to address these challenges by providing a centralized digital platform.

The application allows customers to browse agricultural products, view product details, manage their shopping carts, and place Cash on Delivery (COD) orders. Customers can also browse agricultural equipment and submit rental booking requests. Owners can manage equipment listings and handle booking-related activities. The application also includes an Admin role for reviewing owner applications and viewing revenue summaries.

The application is developed using Java for Android application logic and XML for designing the user interface. Firebase Authentication supports user authentication, while Cloud Firestore stores and retrieves application data.

FarmHub aims to reduce manual effort, improve information accessibility, and provide an organized method of managing agricultural equipment rental and product shopping. Future enhancements may include online payments, notifications, location-based services, and delivery tracking.

TABLE OF CONTENTS

Sr No	Content	Page No
1	Introduction	4
2	Problem statement	10
3	Scope Of The Projet	13
4	Existing System	15
5	Requirements	19
6	Technologies Used	20
7	Implementation Product	22
8	Team Responsibilities And Work Distribution	25
9	Testing And Result	28
10	UML Diagrams	37
11	Screenshot Of App Flow	43
12	Future Enhancement	60
13	References	61

Introduction

Introduction

Agriculture is an important part of the economy and provides food, employment, and raw materials. Farmers require agricultural products such as seeds, fertilizers, tools, and other farming supplies. They may also need equipment such as machines and tools for particular farming activities.

Finding products and arranging equipment rentals can involve contacting different suppliers or equipment owners. Information about availability, prices, rental periods, and booking terms may be spread across different sources. This can make the process less convenient for users.

The use of mobile applications provides an opportunity to make agricultural services easier to access. A mobile application can present information in an organized format and allow users to perform common tasks from their smartphones.

FarmHub is developed with this purpose in mind. It combines an agricultural product marketplace and an equipment rental system in one Android application. It provides functionality for browsing products, managing cart items, placing product orders, viewing equipment, managing equipment listings, and handling rental bookings.

The application also includes role-based access for Customers, Owners, and Admins. Customers can purchase products and request equipment rentals, Owners can manage equipment listings and booking activities, and Admins can review owner applications and view revenue summaries.

Background of the Project

Agricultural activities require both consumable products and physical equipment. Products may be purchased regularly, while some types of equipment may be needed only for a limited period. Purchasing every piece of equipment may not be

practical for every farmer, particularly when the equipment is expensive or required only occasionally.

A digital platform can help organize these activities by providing product information and equipment listings in a common application. Users can view the available information and proceed through the supported shopping or booking workflow.

FarmHub was developed as an academic Android project to explore how these services can be combined. It also provides practical experience with Android activities, Java classes, repositories, authentication, cloud data storage, and role-based application navigation.

Overview of FarmHub

FarmHub consists of three user roles and two main functional areas.

Product Marketplace

This area supports product catalogue browsing, product details, cart management, and product order handling. Customers can browse agricultural products, add items to their carts, and place COD orders.

Agricultural Equipment Rental

This area supports equipment catalogue browsing, equipment details, adding and editing equipment listings, owner equipment management, rental booking records, booking lists, and inspection-related workflows.

Administration

The Admin module supports the review of owner applications and the display of revenue-related information, including product commission, rental commission, total commission, and order and booking counts.

Need for the Project

The main need for FarmHub is to provide a single mobile application for two related agricultural activities: product shopping and equipment rental.

The project aims to organize information and workflows that might otherwise be handled separately. It provides a structured way to display products and equipment and to maintain order and booking records in a cloud database.

The Admin module also provides a way to manage owner applications and access revenue summaries through the application.

Motivation

The motivation behind FarmHub is to explore a practical use of mobile technology in agriculture. Rather than developing a general shopping application alone, the project focuses on agricultural products and equipment-related activities.

The project also demonstrates how authentication, cloud storage, and role-based navigation can be combined to support different users within one application.

Problem Statement

Description of the Problem

Farmers and other agricultural users require products and equipment to perform agricultural work. However, product shopping and equipment rental may be managed through different suppliers and processes. Users may need to spend time searching for product information, comparing available options, and contacting equipment owners.

Equipment is not always required throughout the year. Some machinery is needed only during specific farming operations or seasons. Purchasing expensive equipment for occasional use may not be practical for every farmer. Rental facilities can provide an alternative when suitable equipment is available.

Managing product orders and equipment bookings also requires organized information. Without a structured application, users may find it difficult to keep track of product selections, order details, rental requests, and equipment information.

Owners also require a convenient way to manage equipment listings and booking requests. Administrators need a way to review owner applications and access the available revenue summary.

Therefore, there is a need for a mobile application that brings agricultural product shopping, equipment rental, and basic administrative functions together in one place.

Main Problems Identified

- Separate information sources: Users may need to search different places for products and rental equipment.
- Time-consuming search: Finding relevant information can take time when details are spread across different sources.
- Equipment ownership cost: Buying equipment for occasional use may require a high initial investment.

- Difficulty managing listings: Equipment owners need a convenient method to add, edit, and manage their equipment information.
- Order organization: Product selections and order details need to be stored in a structured manner.
- Booking management: Rental bookings require information such as equipment details, rental duration, charges, and booking status.
- Owner application management: Owner applications need to be reviewed before the relevant access is granted.
- Limited centralization: Product shopping and equipment rental may otherwise require separate applications or processes.

Proposed Solution

FarmHub proposes an Android application that provides product shopping and equipment rental features through a single interface.

Customers can browse agricultural products, view details, use the shopping cart, and place COD orders. They can also browse equipment and participate in the rental booking process.

Owners can manage equipment listings and handle booking-related tasks supported by the application.

Admins can review pending owner applications and approve or reject them. The Admin can also view the available revenue summary.

Firebase Authentication supports user authentication, while Cloud Firestore stores relevant application data.

Expected Outcome

The expected outcome is an Android application that demonstrates:

- Product catalogue and product detail viewing.
- Cart management and COD order placement.
- Equipment catalogue and equipment detail viewing.

- Equipment listing creation and editing by Owners.
- Rental booking and booking-status management.
- Owner application approval and rejection by Admin.
- Revenue-summary viewing by Admin.
- Cloud-based data storage and role-based navigation.

The project aims to provide a useful foundation for a more complete agricultural marketplace in the future.

Scope of the Project

The main objective of FarmHub is to develop an Android application that simplifies agricultural equipment rental and agricultural product shopping through a centralized digital platform.

Specific Objectives

1. To develop a user-friendly Android application.
2. To provide user registration and login functionality.
3. To display agricultural equipment and its relevant details.
4. To allow customers to submit equipment rental requests.
5. To display agricultural products in an organized catalogue.
6. To provide shopping cart and order functionality.
7. To store user, equipment, product, rental, and order information in a database.
8. To implement role-based navigation for Customer, Owner, and Admin.
9. To provide owner equipment listing and booking management.
10. To provide administrative functions for reviewing owner applications and viewing revenue summaries.
11. To reduce dependence on scattered manual records.
12. To design a system that can be expanded with additional features in the future.

Expected Outcomes

The expected outcome is a working Android application that demonstrates the main equipment rental and product shopping workflows.

The system provides a user interface, organized information, authentication, and communication with Firebase services for the implemented features.

The application also provides administrative functions for owner application review and revenue-summary viewing.

Users of the System

FarmHub supports three main user roles.

Customer (User)

The Customer can browse agricultural products, view product details, manage the shopping cart, place COD orders, and access equipment rental features. The Customer can also view order and booking information.

Owner

The Owner can manage equipment listings, add and edit equipment information, and handle booking-related activities supported by the application. Owner access and available operations depend on the application's role and approval workflow.

Admin

The Admin can review pending owner applications, approve or reject applications, and view the available revenue summary. The Admin has a separate administrative role from the Customer and Owner.

Project Boundaries

The project is limited to the features implemented in the current Android application.

FarmHub supports product shopping, COD orders, equipment listings, rental bookings, owner application management, and revenue-summary viewing.

The project does not include a completed online payment gateway. Features such as delivery tracking, advanced agricultural analytics, and additional notification services may be considered for future development.

The availability of individual screens and actions depends on the application build and the data configured in Firebase. Any feature not implemented in the current application should be described under Future Enhancements.

Existing System

Nowadays, several applications provide agricultural products and farming services. Some applications focus on selling agricultural products, while others provide farming advice or equipment rental services. However, their features and services differ from one platform to another.

BigHaat: BigHaat provides agricultural products such as seeds, fertilizers, crop protection products, and farming tools. It mainly focuses on helping farmers purchase agricultural supplies online.

DeHaat: DeHaat provides agricultural inputs, farming advice, and services related to agricultural markets. It aims to support farmers throughout different stages of farming.

FARMS App: The FARMS application supports the hiring of farm machinery and the buying and selling of agricultural equipment.

JFarm Services: JFarm Services connects farmers with tractor and other farm equipment rental services.

Limitations of the Existing System

Different agricultural applications focus on different services. Some mainly provide agricultural products, while others focus on farming advice or machinery rental. Farmers may need to use multiple applications to access different services.

FarmHub aims to bring agricultural product shopping, equipment rental, and basic administrative functions together in a single Android application.

Proposed System: FarmHub

FarmHub is an Android application developed to make agricultural product shopping and equipment rental more convenient.

The application supports three roles: Customer, Owner, and Admin.

Customers can browse agricultural products, view product details, add products to the cart, and place orders using Cash on Delivery (COD). They can also browse available farming equipment and make rental bookings.

Owners can manage equipment listings, review booking requests, and handle supported booking-related activities.

Admins can review pending owner applications, approve or reject applications, and view revenue summaries.

The application uses Firebase Authentication for user authentication and Cloud Firestore to store and manage application data.

Comparison Between Existing Applications and FarmHub

Use the following table in place of your existing comparison table.

Feature	BigHaat	DeHaat	FARMS App	JFarm Services	FarmHub
Agricultural products	Yes	Yes	Not its main focus	Not its main focus	Yes
Farming advice	Related services	Yes	Not its main focus	Not its main focus	Not included
Equipment rental	Not its main focus	Not its main focus	Yes	Yes	Yes
Product catalogue	Yes	Available services vary	Equipment-focused	Rental-focused	Yes

Shopping cart and COD orders	Depends on platform	Depends on platform	Not its main focus	Not its main focus	Yes
Owner equipment management	Varies	Varies	Machinery services	Rental services	Yes
Admin owner approval	Varies	Varies	Varies	Varies	Yes
Revenue summary	Varies	Varies	Varies	Varies	Available to Admin

Note: This comparison describes the general focus of the listed platforms. Their features may vary according to the application version and available services.

Advantages of the Proposed System

- Combines agricultural product shopping and equipment rental in one application.
- Provides separate workflows for Customers, Owners, and Admins.
- Allows customers to browse products and place COD orders.
- Allows owners to manage equipment listings and booking requests.
- Supports owner application approval and rejection.
- Provides an Admin revenue summary.
- Uses Firebase services for authentication and cloud data storage.

Conclusion

FarmHub provides a convenient way to access agricultural products and equipment rental through one Android application. It combines product browsing, cart management, COD ordering, equipment listings, and rental bookings.

The application also includes administrative functionality for owner application management and revenue-summary viewing. Its purpose is to simplify access to

agricultural services for customers and equipment owners while providing basic administrative control.

Requirements

Hardware Requirements

- Computer or laptop for application development.
- Android smartphone or Android emulator for application testing.
- Internet connection for Firebase authentication and cloud data operations.

Software Requirements

Software	Purpose
Fedora Linux or another supported operating system	Development environment
Android SDK	Android application development
Java	Application programming
XML	User-interface layout design
Firebase Authentication	User registration and login
Cloud Firestore	Application data storage
Gradle	Build automation and dependency management
Android Debug Bridge (ADB)	Installing and testing the application on a connected device
Visual Studio Code	Editing and managing project source files

TECHNOLOGIES USED

Java

Java is the main programming language used to implement application logic, manage user actions, and handle navigation between screens.

XML

XML is used to design application layouts, including buttons, text fields, labels, and other user-interface components.

Android SDK

The Android SDK provides the tools, libraries, and APIs required to develop and run the Android application.

Firebase Authentication

Firebase Authentication supports user registration and login.

Cloud Firestore

Cloud Firestore stores and retrieves application data related to users, products, equipment, carts, orders, bookings, and owner applications.

Gradle

Gradle manages the application build process and project dependencies.

Android Debug Bridge (ADB)

ADB is used to install, run, and debug the application on a connected Android device.

Visual Studio Code

Visual Studio Code is used to edit and manage project source files.

Platform: Android mobile application

Architecture: Activity-based Android application with repository classes for data operations and Firebase services for authentication and cloud data storage.

Implementation Details

Implementation is the process of converting the system design into a working application. FarmHub is an Android application developed using Java and XML, with Firebase services supporting authentication and cloud data storage.

Android Application Structure

The application contains activity classes for screens, model classes for data, repository classes for database operations, and supporting classes for image loading and navigation.

This structure helps organize the application into separate components and makes the source code easier to maintain.

Product Implementation

The product module allows customers to browse agricultural products, view product details, and select items for purchase.

Classes such as `Product.java`, `ProductCatalogActivity.java`, `ProductDetailsActivity.java`, and `ProductRepository.java` support product information, screen display, and product-related data operations.

Cart and Order Implementation

Customers can add products to their cart and place orders using Cash on Delivery.

Cart items and order details are managed through the application's data layer and Cloud Firestore. Order information may include delivery details, total amount, ordered items, and order status.

Online payment processing is not implemented.

Equipment Implementation

The equipment module allows customers to view available farming equipment.

Owners can add new equipment listings, edit existing information, and manage their equipment.

Classes such as `Equipment.java`, `EquipmentCatalogActivity.java`, and `EquipmentRepository.java` support these functions.

Booking Implementation

The booking module allows customers to request equipment rentals.

`Booking.java` represents booking information, while `BookingRepository.java` handles booking-related data operations.

Owners can review booking requests and make supported booking decisions. The application also supports recording equipment return-inspection details.

Authentication and Role Management

FarmHub uses Firebase Authentication for user sign-in and Cloud Firestore for data storage.

The application includes Customer, Owner, and Admin roles. Role-based access logic determines the screens and operations available to each role.

Customers can shop and book equipment. Owners manage equipment listings and booking activities. Admins can review owner applications and access the revenue summary.

Admin Implementation

The Admin module provides administrative functions separate from the Customer and Owner workflows.

`AdminApprovalActivity.java` supports the owner-application approval workflow. The Admin can review pending applications and approve or reject them.

`RevenueActivity.java` provides a revenue summary that includes product commission, rental commission, total commission, and order and booking counts.

The actual operations available depend on the implemented application screens and configured Firebase data.

Testing and Execution

The application can be installed and run on a compatible Android emulator or physical device.

Testing should cover login, product browsing, cart operations, COD orders, equipment management, rental bookings, owner approval, and revenue-summary viewing.

Screenshots of successfully tested features should be included in the report.

Team Responsibilities And Work Distribution

Teamwork is an important part of software development. A project can be divided into modules so that each team member can focus on a particular area while coordinating with the rest of the team.

FarmHub covers product marketplace functionality, equipment rental and booking, authentication, role management, administration, integration, and testing.

The following allocation should reflect the team's actual contributions.

Product Marketplace

Team Member Name: Jagruti Gadekar

The product marketplace responsibility includes product browsing, product details, cart operations, product images, and order-related functionality.

Important files include:

- `Product.java`
- `ProductCatalogActivity.java`
- `ProductDetailsActivity.java`
- `ProductRepository.java`
- `ProductImageLoader.java`
- `CartActivity.java`
- `ProductOrdersActivity.java`
- `SellerProductsActivity.java`

The work involves displaying product information, organizing product data access, supporting cart operations, and connecting product selection with order placement.

Equipment Rental and Booking

Team Member Name: Dhanashri Ghumate

The equipment rental and booking responsibility includes equipment listings, equipment details, owner equipment management, and rental booking activities.

Important files include:

- `Equipment.java`
- `EquipmentCatalogActivity.java`
- `EquipmentDetailsActivity.java`
- `EquipmentRepository.java`
- `AddEquipmentActivity.java`
- `EditEquipmentActivity.java`
- `OwnerEquipmentActivity.java`
- `Booking.java`
- `BookingRepository.java`
- `BookingListActivity.java`
- `BookingInspectionActivity.java`

The work involves managing equipment information, displaying available equipment, supporting listing changes, and handling booking-related data.

Authentication, Role Management, Admin, Integration, and Testing

Team Member Name: Magnus Rodrigues

This responsibility includes authentication, role-aware navigation, administrative functions, integration between modules, and testing.

Important files include:

- `MainActivity.java`
- `HomeActivity.java`
- `RoleAccess.java`
- `RoleDestinationActivity.java`

- `AdminApprovalActivity.java`
- `RevenueActivity.java`
- Firebase configuration and integration files.

The work involves connecting application screens, managing role-based navigation, integrating the product and rental modules, supporting owner application review, and checking the Admin revenue summary.

TESTING AND RESULTS

Software testing is performed to check whether an application behaves as expected. It helps identify errors and verifies that different modules work together correctly.

FarmHub requires testing of authentication, product browsing, cart operations, order placement, equipment listings, rental bookings, owner application management, and Admin functions.

Objectives of Testing

The main objectives are:

- To check whether the application launches correctly.
- To test authentication and role-based navigation.
- To verify product catalogue and product detail screens.
- To check cart operations and COD order placement.
- To test equipment listings and rental bookings.
- To verify owner application approval and rejection.
- To check the Admin revenue summary.
- To identify errors in data loading and saving.
- To check how the application handles invalid input and network problems.

Unit Testing

Unit testing checks a small part of a program independently. For FarmHub, possible unit-testing targets include validation logic, calculations, status handling, and utility methods.

For example, a test could check whether a total is calculated correctly for a known set of cart items. Another test could verify how the application handles an empty input field.

Automated unit tests should be mentioned only if they were actually written and executed.

Integration Testing

Integration testing checks whether different parts of the application work together.

Examples include:

- Product catalogue to product detail navigation.
- Product selection to cart storage.
- Cart data to COD order placement.
- Equipment catalogue to equipment details.
- Owner equipment management to stored equipment data.
- Booking list to booking updates.
- Authentication to role-based navigation.
- Admin approval actions to owner application status updates.
- Admin revenue screen to stored order and booking information.

Functional Testing

Functional testing checks whether the available features behave according to their requirements.

Test ID	Feature	Test Action	Expected Result
TC-01	Application launch	Open FarmHub	Application opens without unexpected errors
TC-02	Authentication	Log in using valid credentials	User is authenticated
TC-03	Product catalogue	Open the product catalogue	Available products are displayed
TC-04	Product details	Select a product	Relevant product details are displayed

TC-05	Shopping cart	Add a product to the cart	Selected product appears in the cart
TC-06	Cart persistence	Reopen the cart	Stored cart items load correctly
TC-07	COD order	Place an order with valid details	Order information is saved
TC-08	Equipment catalogue	Open equipment catalogue	Available equipment is displayed
TC-09	Equipment details	Select equipment	Equipment details are displayed
TC-10	Add equipment	Submit valid equipment information as Owner	Listing is saved
TC-11	Edit equipment	Modify an existing listing	Changes are saved
TC-12	Booking list	Open the relevant booking list	Relevant booking records are displayed
TC-13	Booking decision	Approve or reject a booking where supported	Booking status is updated
TC-14	Inspection	Record equipment inspection details	Inspection information is saved
TC-15	Owner application	Open pending owner applications as Admin	Pending applications are displayed
TC-16	Owner approval	Approve or reject an application	Application status is updated

TC-17	Revenue summary	Open the Admin revenue screen	Available commission and count information is displayed
TC-18	Invalid input	Submit incomplete or invalid information	Input is handled appropriately
TC-19	Network interruption	Interrupt connectivity during an operation	Application handles the failure appropriately

The actual-result column should be completed after testing. Do not mark every test as passed unless the corresponding test has actually been performed.

Negative Testing

Negative testing checks how the application behaves when a user provides invalid input or when an operation cannot be completed.

Examples include:

- Submitting a form with missing required information.
- Opening a record that no longer exists.
- Loading data without an internet connection.
- Attempting an operation without the required authentication state.
- Trying to modify information that should not be accessible to the current user.
- Attempting to access Admin functions from a Customer account.

Security Testing

Security testing checks whether application data is protected.

Important checks include:

- Whether Firestore security rules restrict access to appropriate records.
- Whether customers can access only the cart data permitted for their accounts.

- Whether owner-only operations are protected against unauthorized users.
- Whether Admin functions are restricted to authorized Admin accounts.
- Whether users can modify order or booking records in unauthorized ways.

Security cannot be confirmed solely by successfully navigating the application. Firestore rules and other access permissions should be reviewed and tested separately.

Testing Results

The final testing results should be based on the application build that is actually submitted.

After testing, replace pending results with the observed outcome, such as Pass or Fail, and include a brief note for any failure.

A final claim about complete functional success should be made only after the test cases have been executed and the observed results recorded.

Software testing is performed to check whether an application behaves as expected. It helps identify errors and verifies that different modules work together correctly.

FarmHub requires testing of authentication, product browsing, cart operations, order placement, equipment listings, rental bookings, owner application management, and Admin functions.

Objectives of Testing

The main objectives are:

- To check whether the application launches correctly.
- To test authentication and role-based navigation.
- To verify product catalogue and product detail screens.
- To check cart operations and COD order placement.
- To test equipment listings and rental bookings.
- To verify owner application approval and rejection.
- To check the Admin revenue summary.

- To identify errors in data loading and saving.
- To check how the application handles invalid input and network problems.

Unit Testing

Unit testing checks a small part of a program independently. For FarmHub, possible unit-testing targets include validation logic, calculations, status handling, and utility methods.

For example, a test could check whether a total is calculated correctly for a known set of cart items. Another test could verify how the application handles an empty input field.

Automated unit tests should be mentioned only if they were actually written and executed.

Integration Testing

Integration testing checks whether different parts of the application work together.

Examples include:

- Product catalogue to product detail navigation.
- Product selection to cart storage.
- Cart data to COD order placement.
- Equipment catalogue to equipment details.
- Owner equipment management to stored equipment data.
- Booking list to booking updates.
- Authentication to role-based navigation.
- Admin approval actions to owner application status updates.
- Admin revenue screen to stored order and booking information.

Functional Testing

Functional testing checks whether the available features behave according to their requirements.

Test ID	Feature	Test Action	Expected Result
TC-01	Application launch	Open FarmHub	Application opens without unexpected errors
TC-02	Authentication	Log in using valid credentials	User is authenticated
TC-03	Product catalogue	Open the product catalogue	Available products are displayed
TC-04	Product details	Select a product	Relevant product details are displayed
TC-05	Shopping cart	Add a product to the cart	Selected product appears in the cart
TC-06	Cart persistence	Reopen the cart	Stored cart items load correctly
TC-07	COD order	Place an order with valid details	Order information is saved
TC-08	Equipment catalogue	Open equipment catalogue	Available equipment is displayed
TC-09	Equipment details	Select equipment	Equipment details are displayed
TC-10	Add equipment	Submit valid equipment information as Owner	Listing is saved
TC-11	Edit equipment	Modify an existing listing	Changes are saved
TC-12	Booking list	Open the relevant booking list	Relevant booking records are displayed

TC-13	Booking decision	Approve or reject a booking where supported	Booking status is updated
TC-14	Inspection	Record equipment inspection details	Inspection information is saved
TC-15	Owner application	Open pending owner applications as Admin	Pending applications are displayed
TC-16	Owner approval	Approve or reject an application	Application status is updated
TC-17	Revenue summary	Open the Admin revenue screen	Available commission and count information is displayed
TC-18	Invalid input	Submit incomplete or invalid information	Input is handled appropriately
TC-19	Network interruption	Interrupt connectivity during an operation	Application handles the failure appropriately

The actual-result column should be completed after testing. Do not mark every test as passed unless the corresponding test has actually been performed.

Negative Testing

Negative testing checks how the application behaves when a user provides invalid input or when an operation cannot be completed.

Examples include:

- Submitting a form with missing required information.
- Opening a record that no longer exists.
- Loading data without an internet connection.
- Attempting an operation without the required authentication state.

- Trying to modify information that should not be accessible to the current user.
- Attempting to access Admin functions from a Customer account.

Security Testing

Security testing checks whether application data is protected.

Important checks include:

- Whether Firestore security rules restrict access to appropriate records.
- Whether customers can access only the cart data permitted for their accounts.
- Whether owner-only operations are protected against unauthorized users.
- Whether Admin functions are restricted to authorized Admin accounts.
- Whether users can modify order or booking records in unauthorized ways.

Security cannot be confirmed solely by successfully navigating the application. Firestore rules and other access permissions should be reviewed and tested separately.

Testing Results

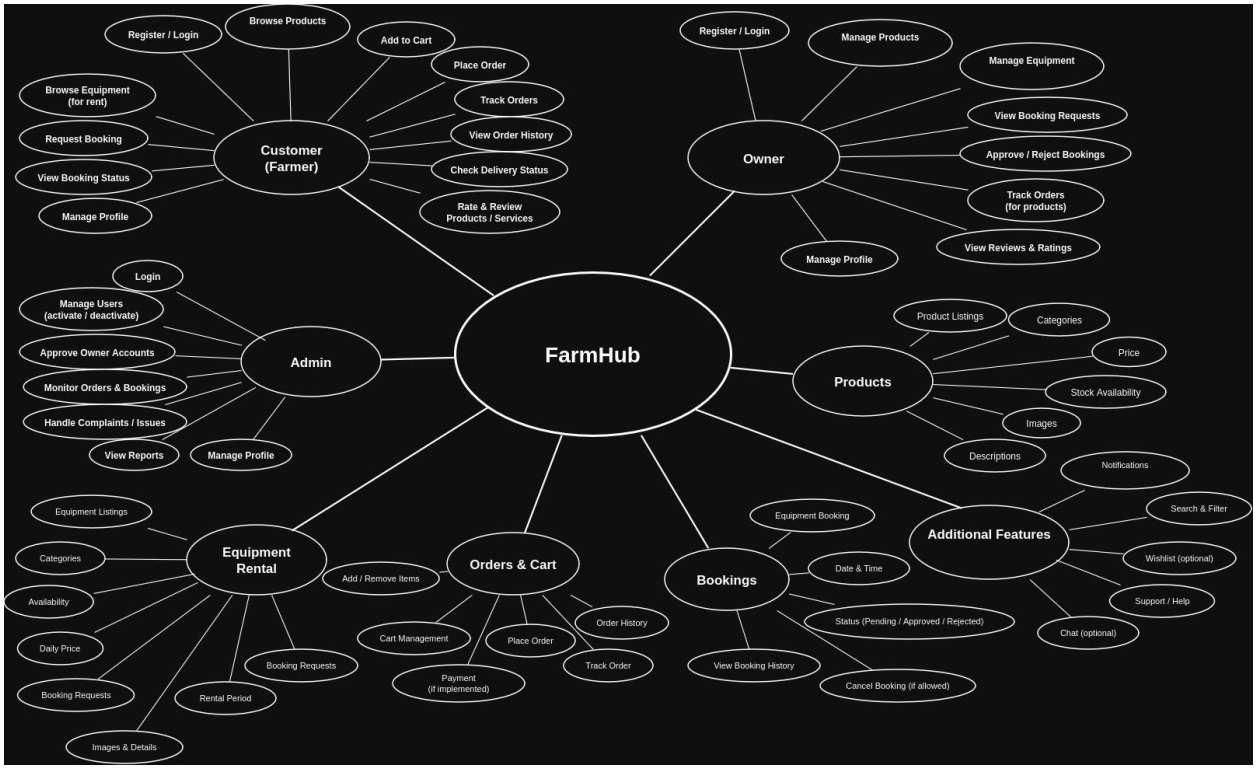
The final testing results should be based on the application build that is actually submitted.

After testing, replace pending results with the observed outcome, such as Pass or Fail, and include a brief note for any failure.

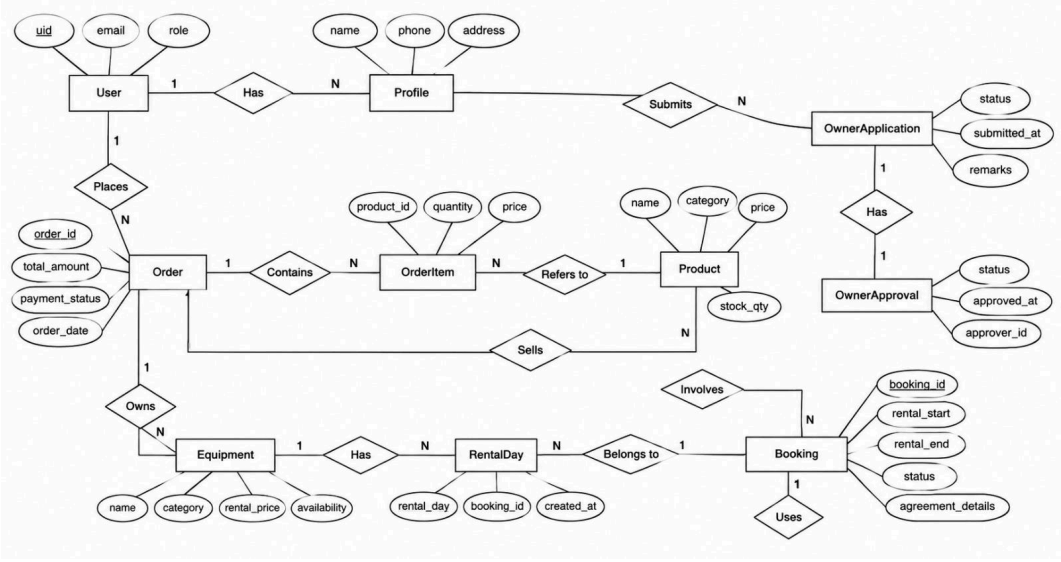
A final claim about complete functional success should be made only after the test cases have been executed and the observed results recorded.

UML DAIGRAMS

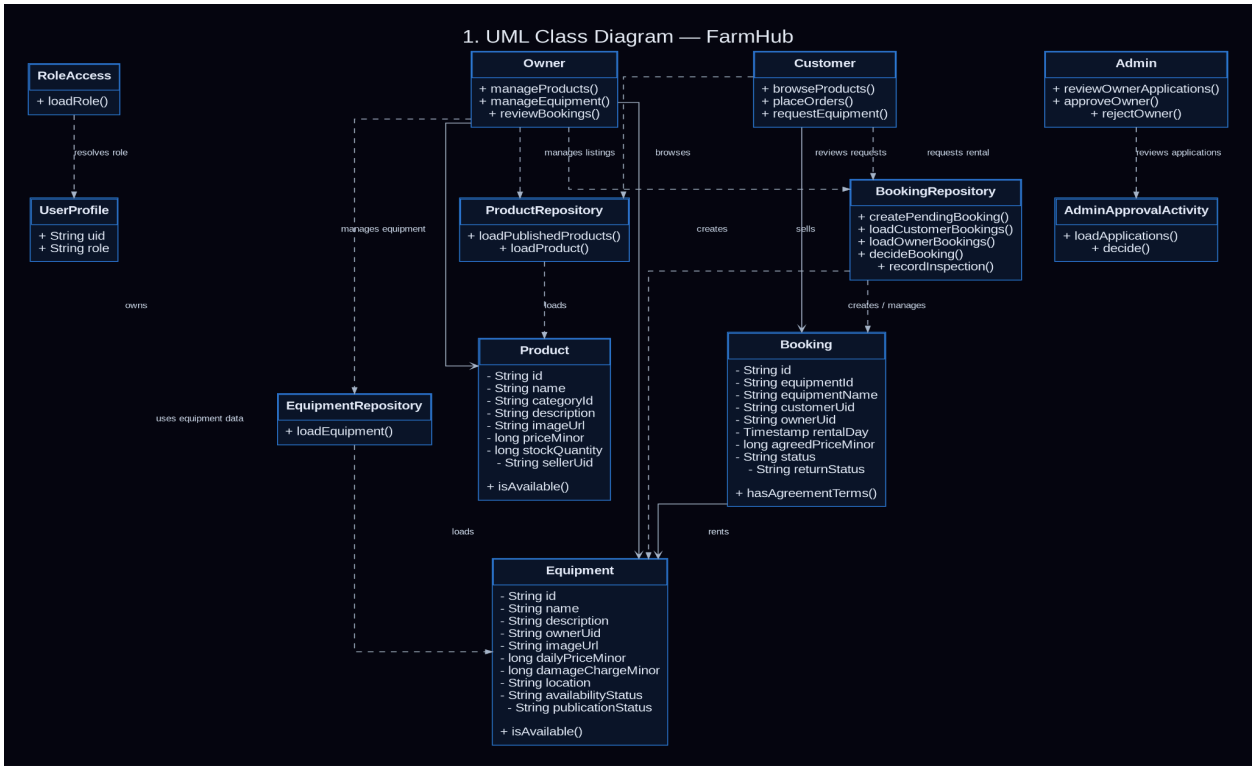
Brainstorming Diagram



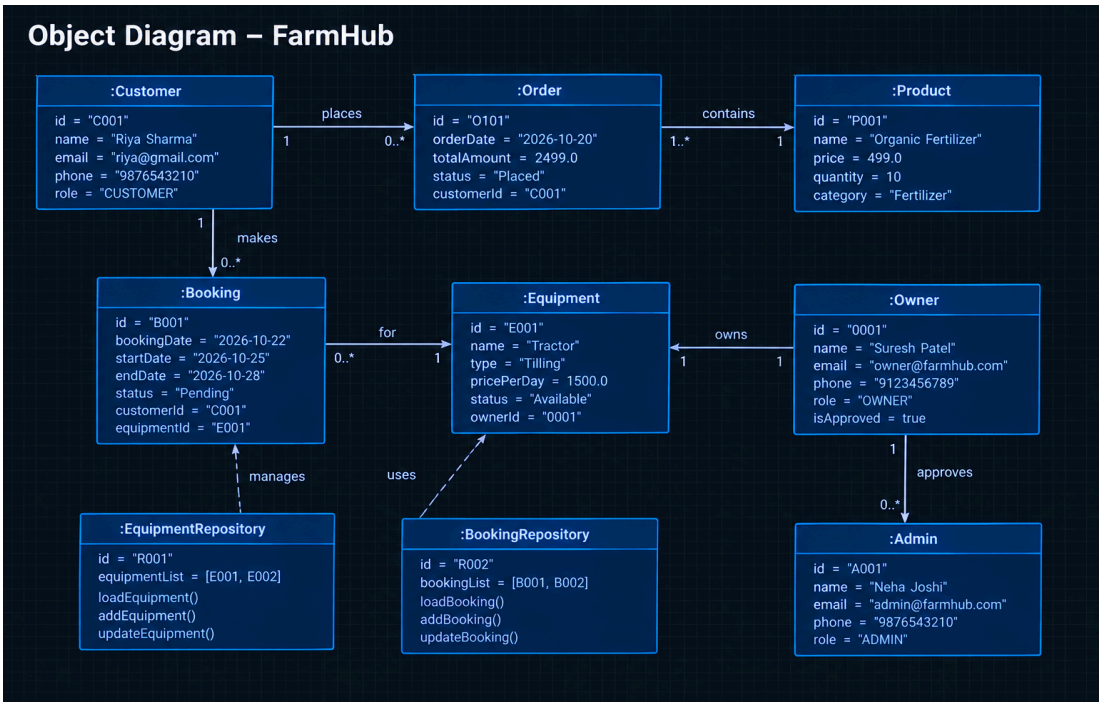
ER Diagram



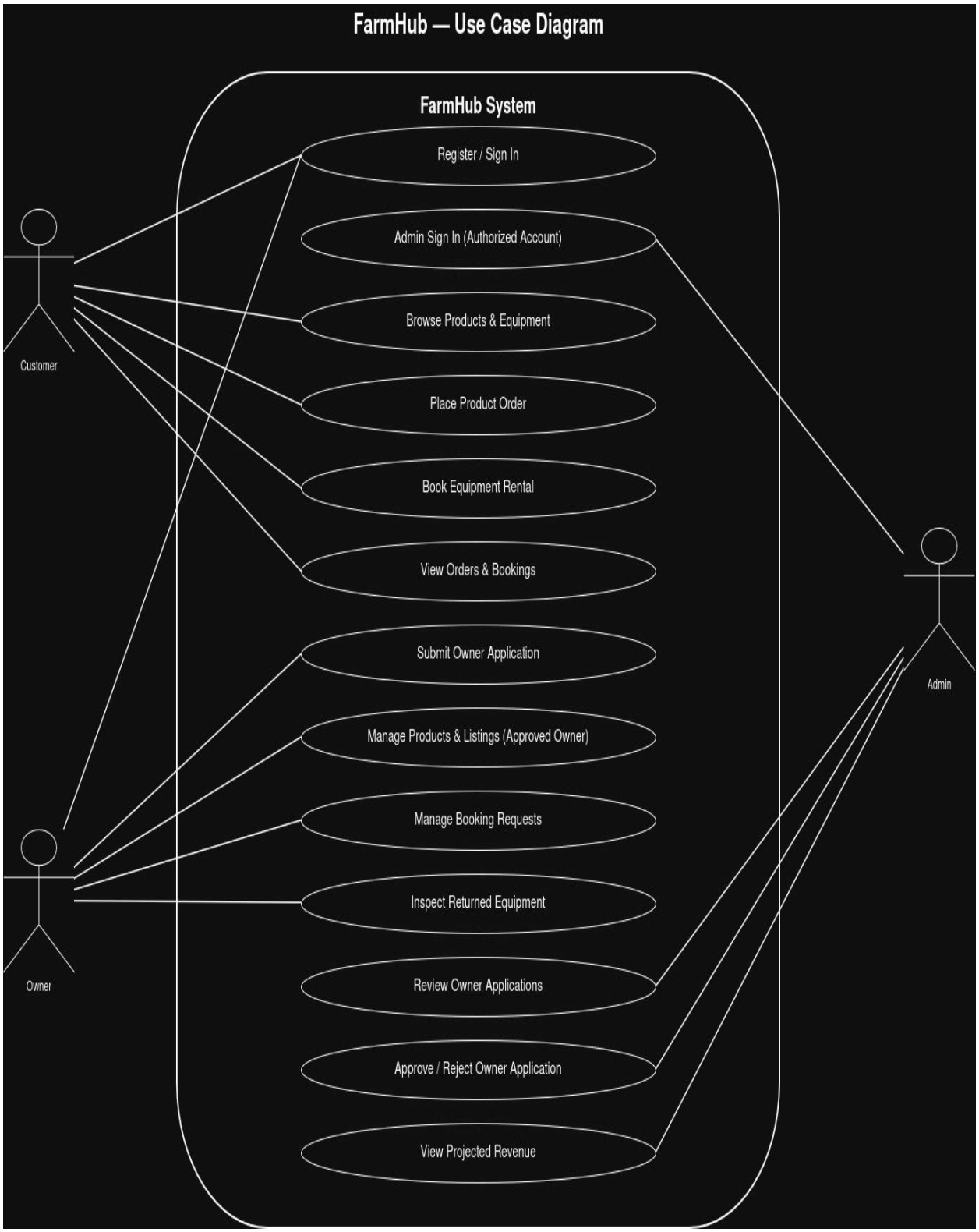
Class Diagram



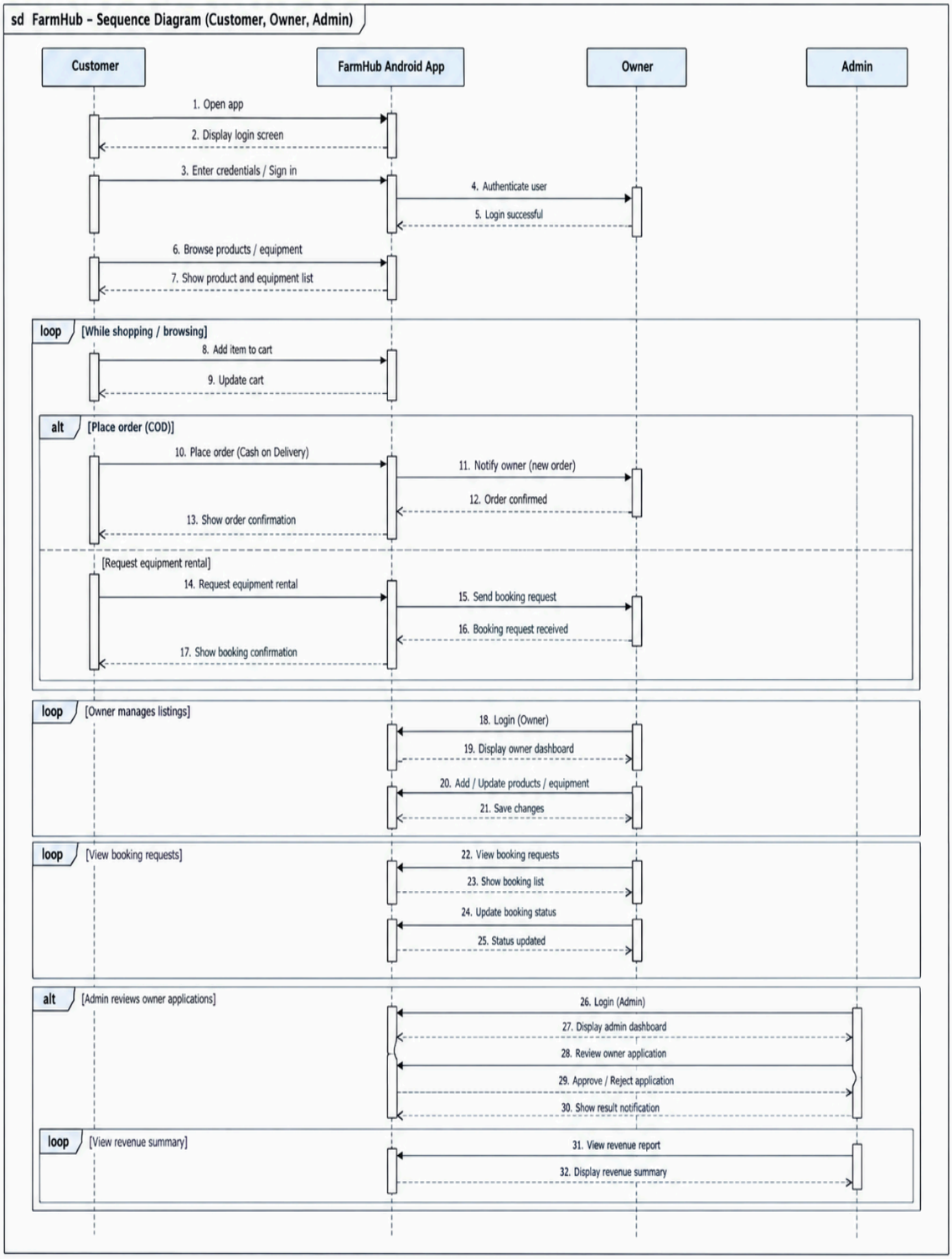
Object Diagram



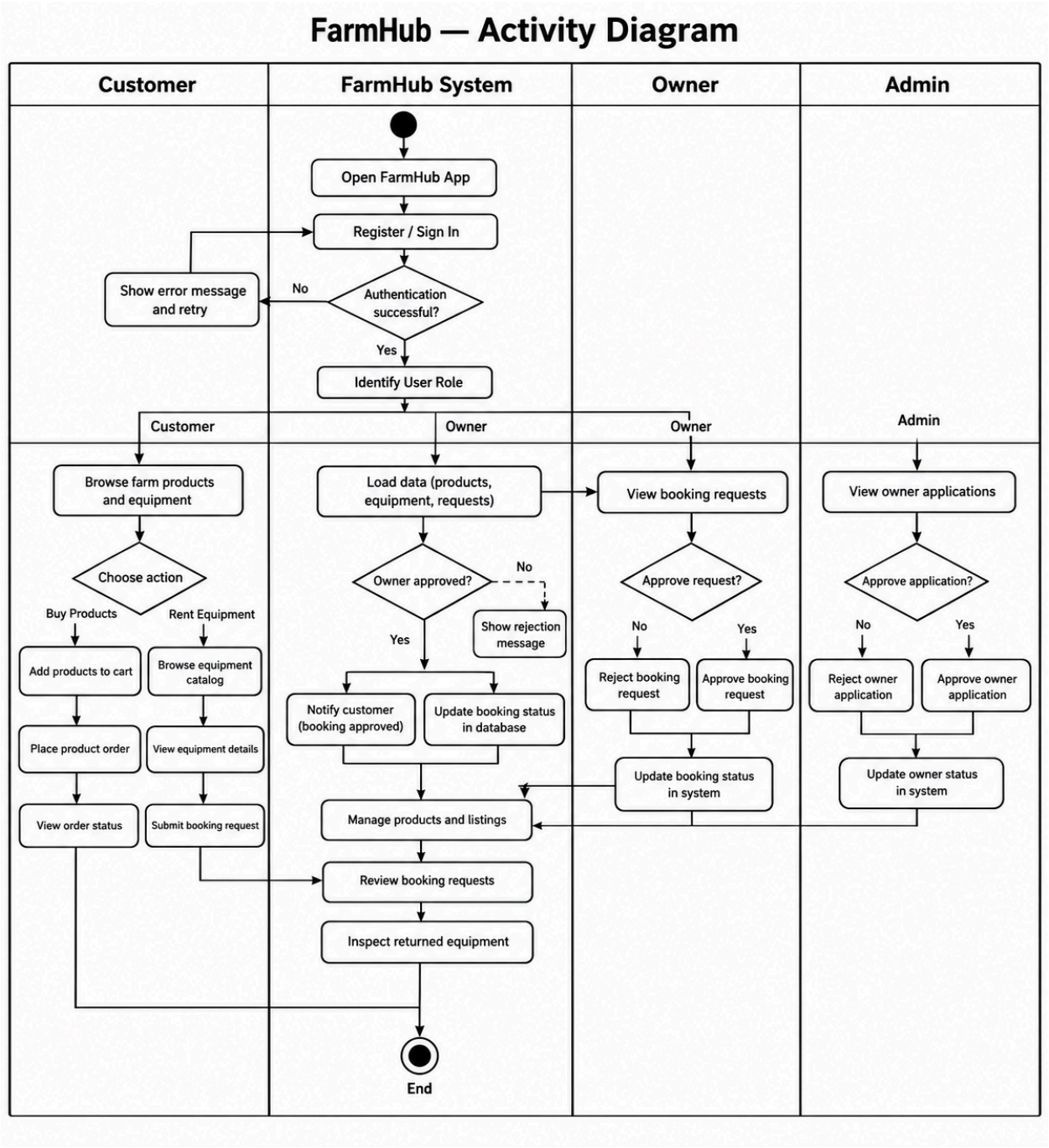
Use Case Diagram


FarmHub — Use Case Diagram
FarmHub System
Register / Sign In
Admin Sign In (Authorized Account)
Browse Products & Equipment
Place Product Order
Book Equipment Rental
View Orders & Bookings
Submit Owner Application
Manage Products & Listings (Approved Owner)
Manage Booking Requests
Inspect Returned Equipment
Review Owner Applications
Approve / Reject Owner Application
View Projected Revenue
Customer
Owner
Admin

Sequence Diagram



Activity Diagram

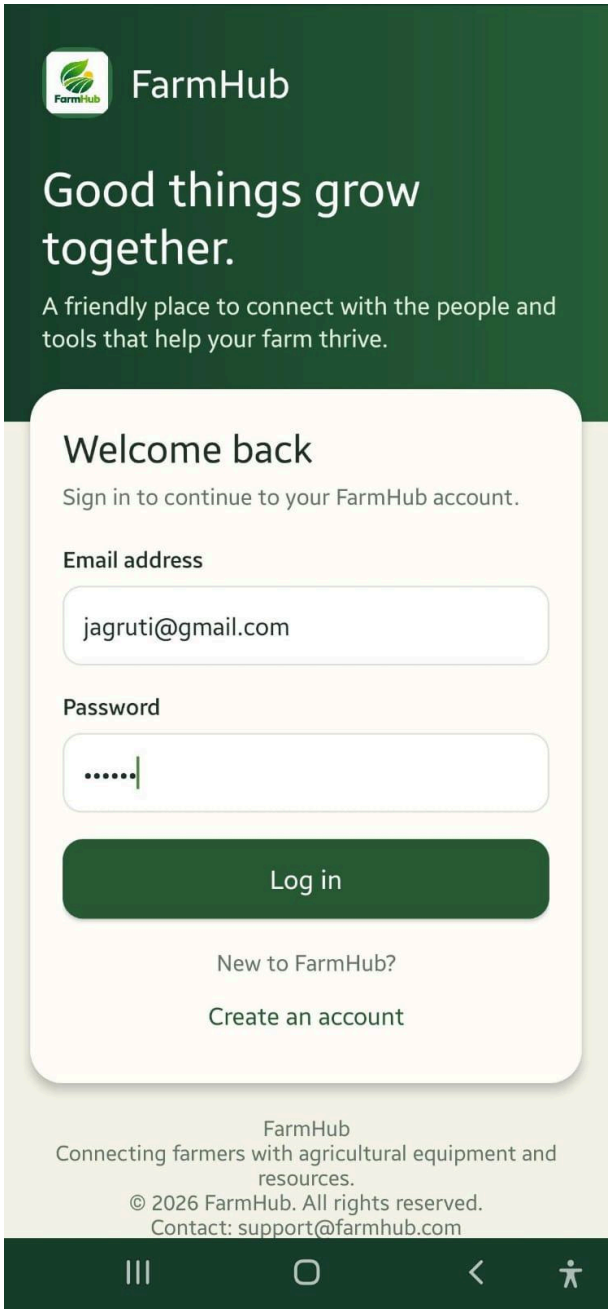


SCREENSHOT OF APP FLOW

Customer Flow

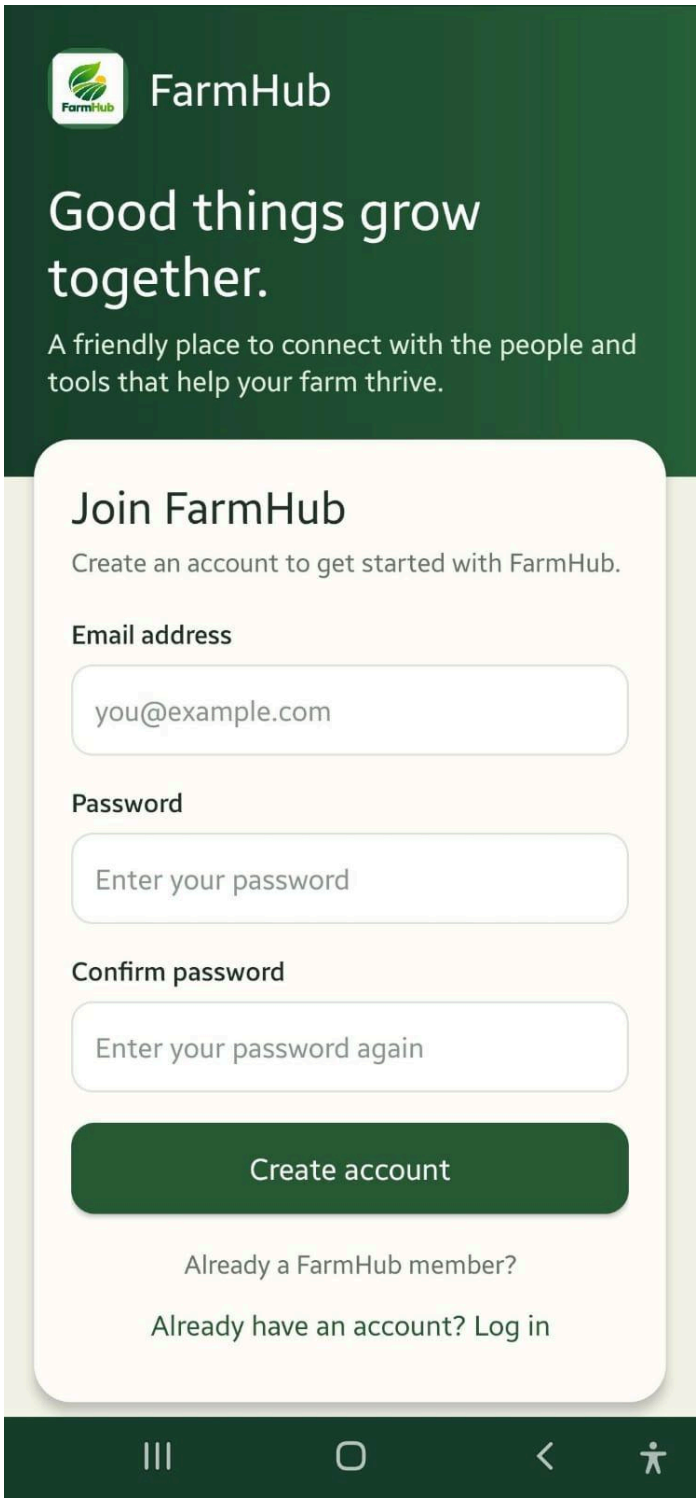
1 . Login Screen

The customer enters their email and password to sign in.



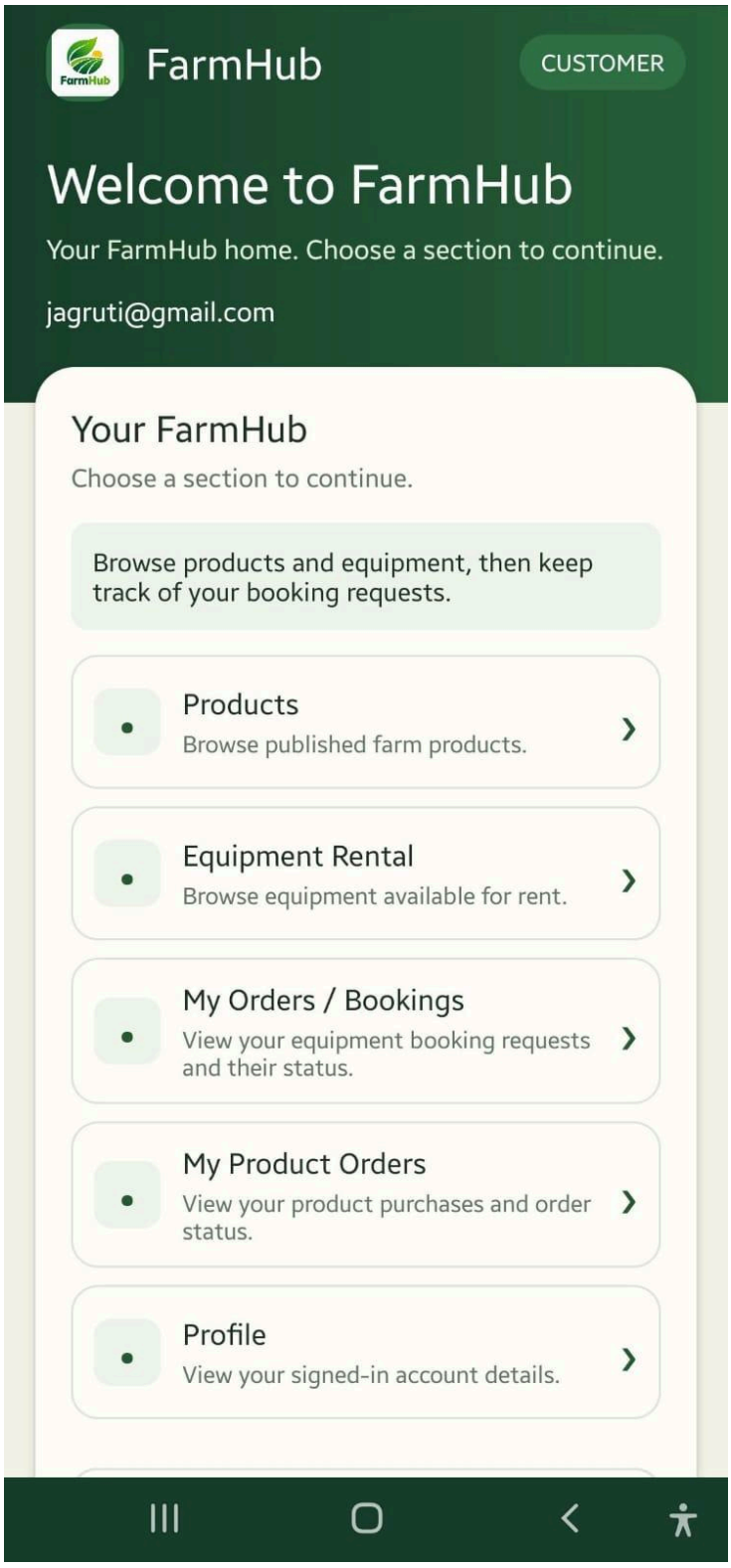
2. Registration Screen

A new user selects the Create Account option, enters their email, password, and confirms the password to register.



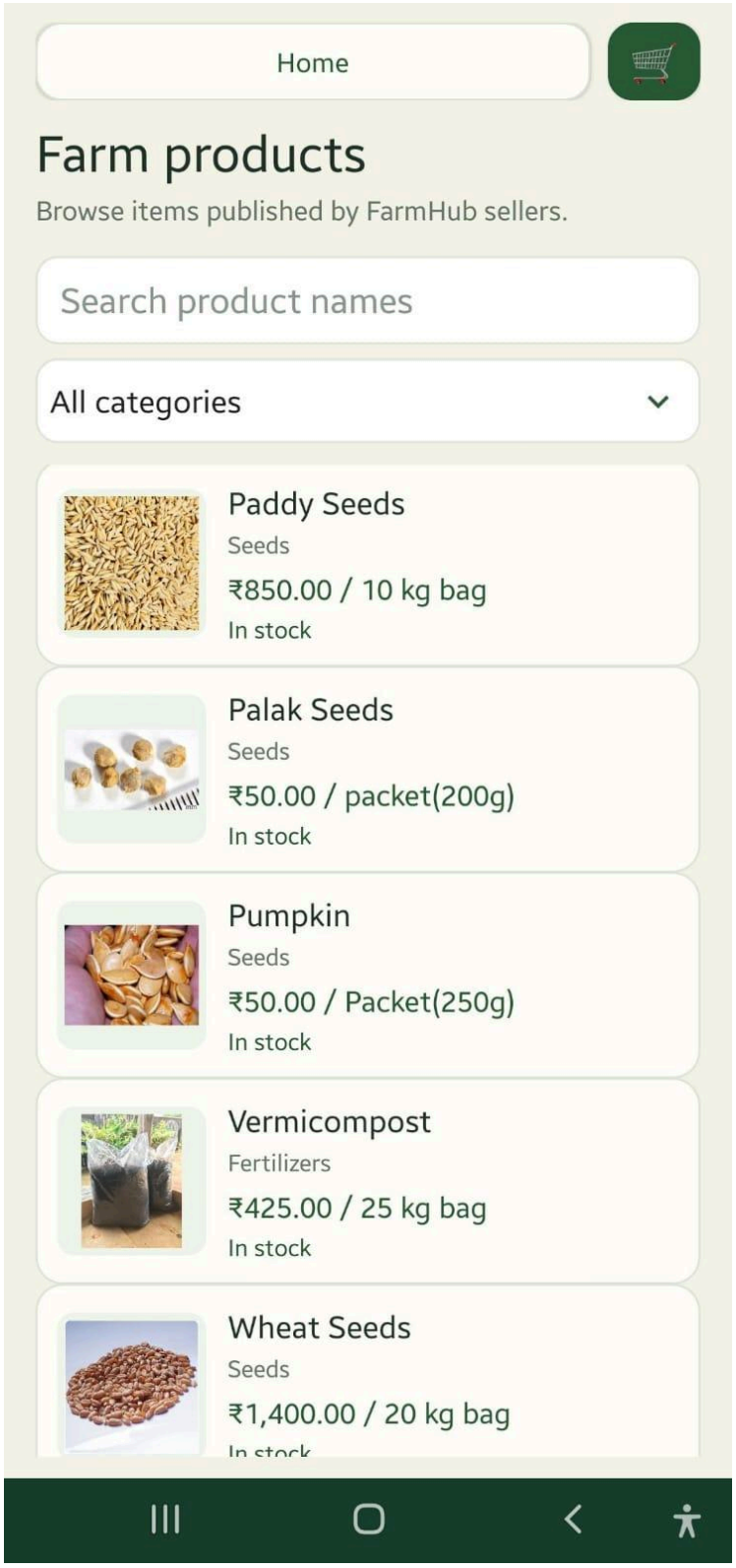
3: Home Screen

The customer accesses product shopping and equipment rental.



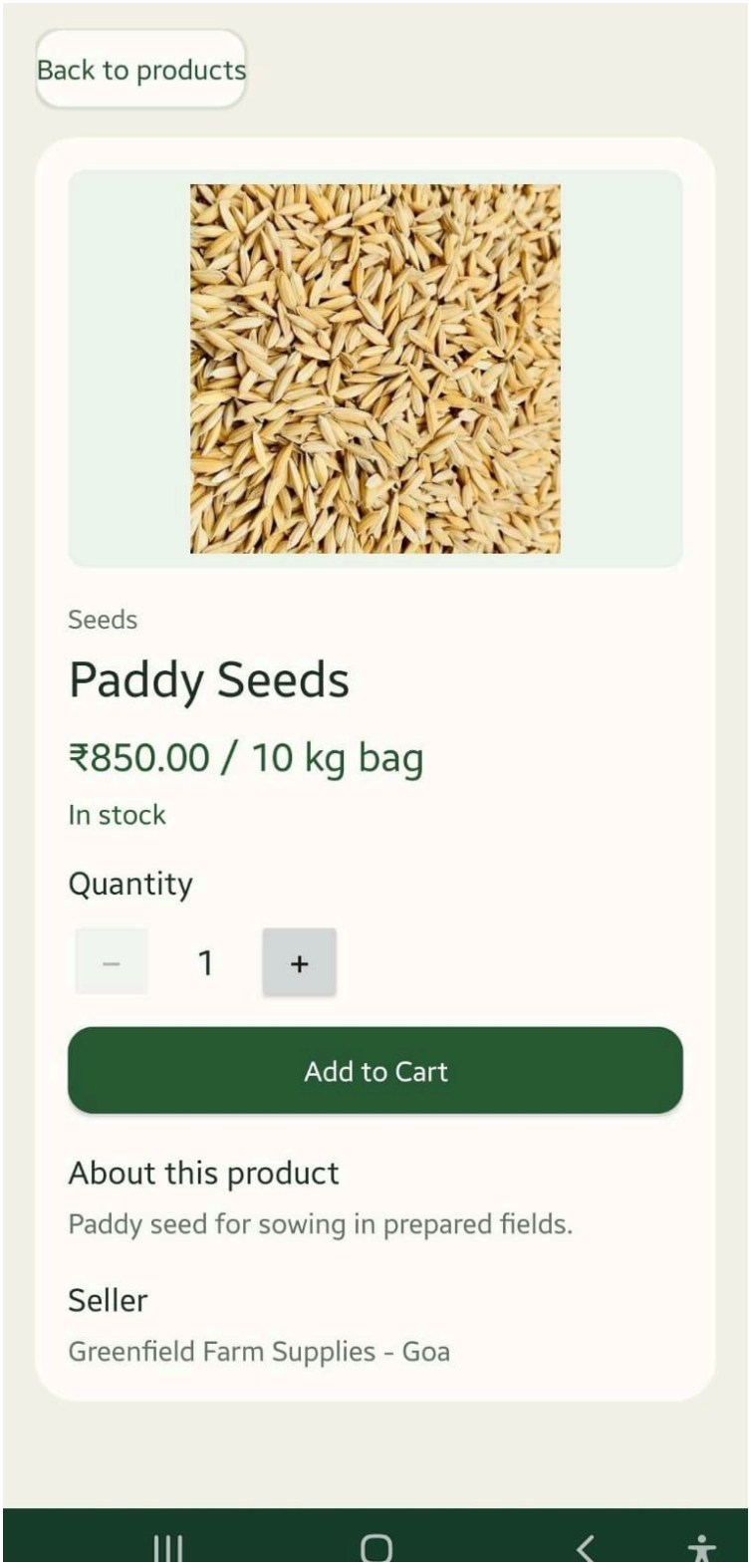
4: Product Catalogue

The customer browses available agricultural products.



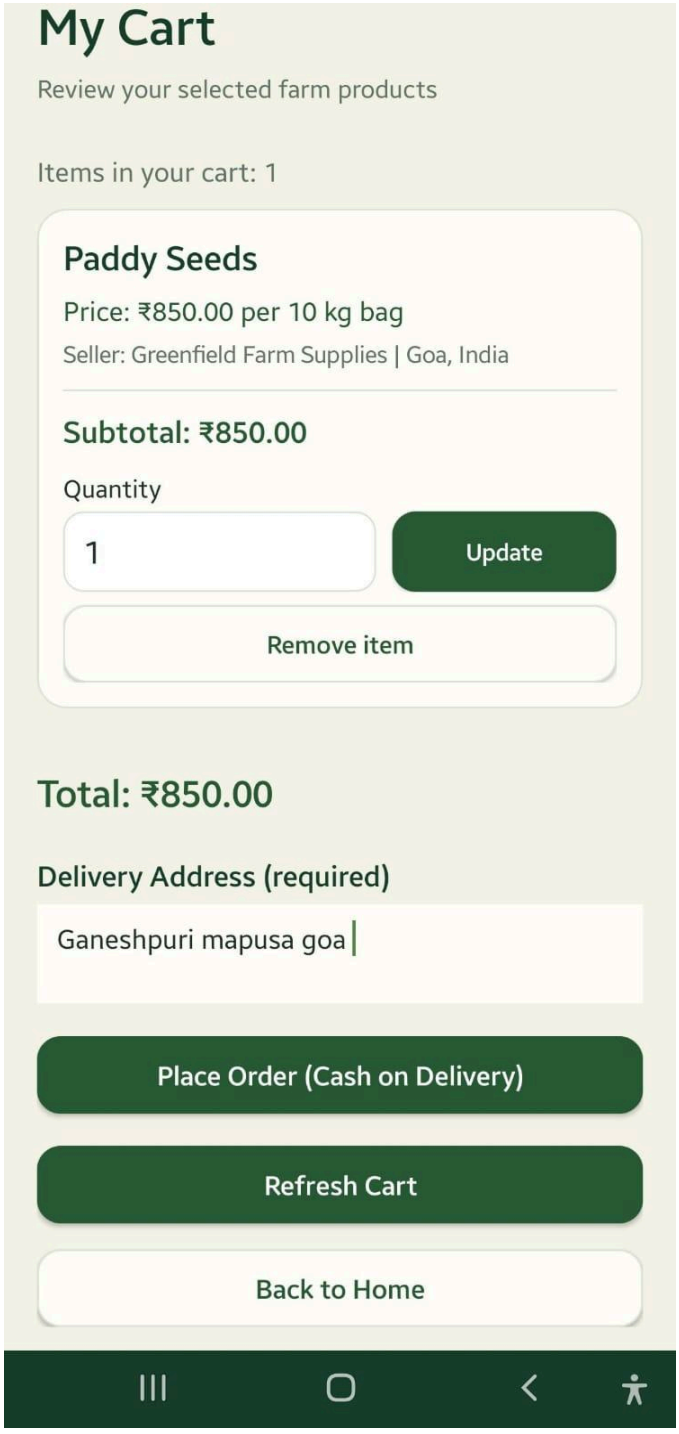
5: Product Details

The customer views the selected product's information.



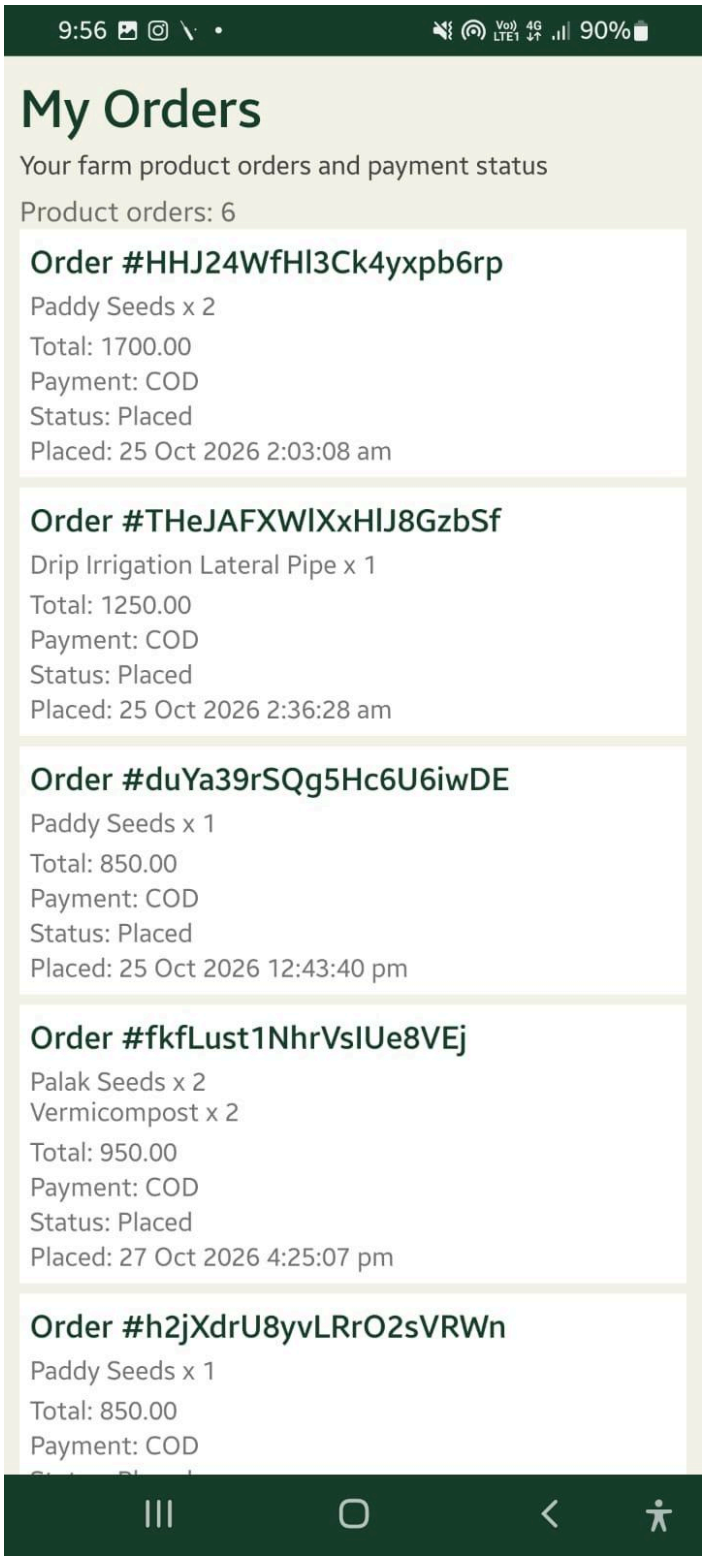
6: Place Order

The customer enters a delivery address and places a COD order.



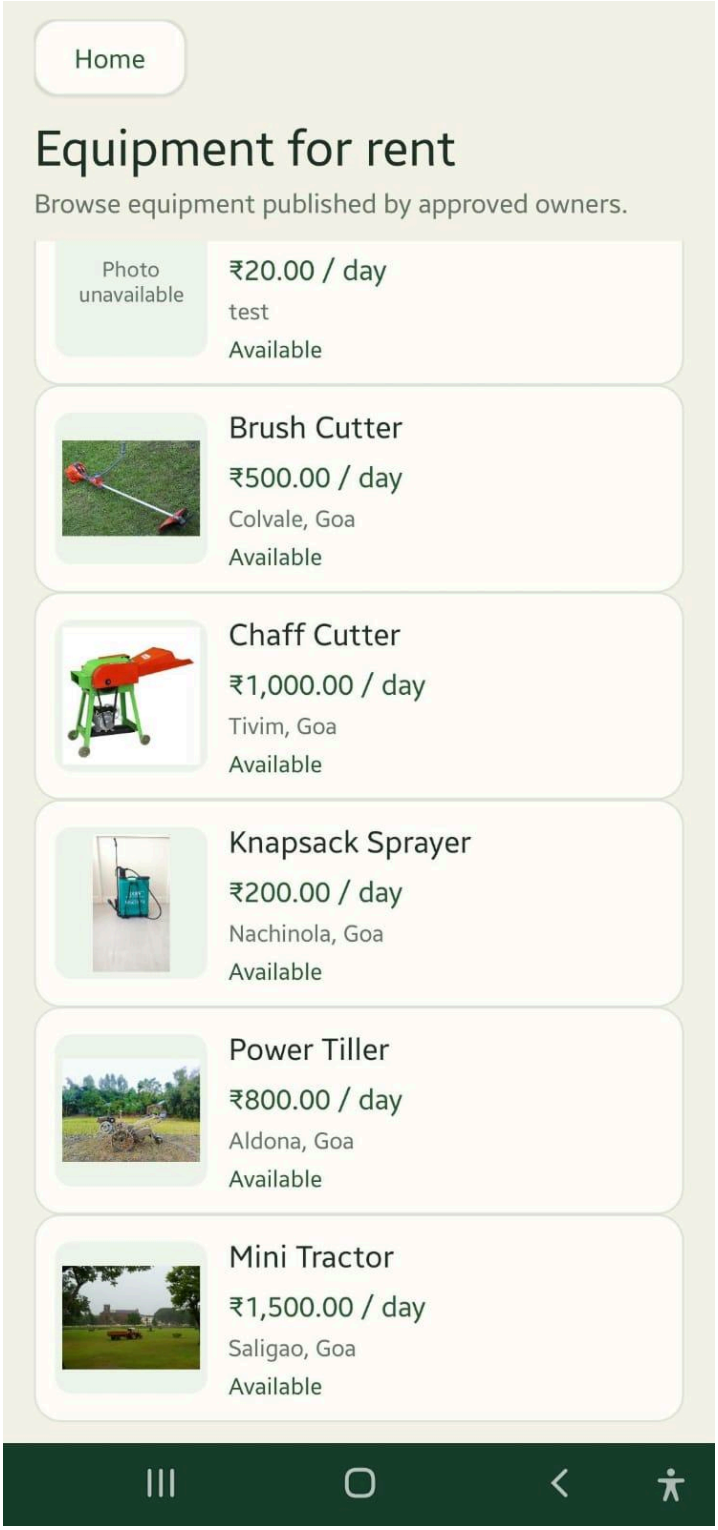
7: My Orders

The customer views placed orders.



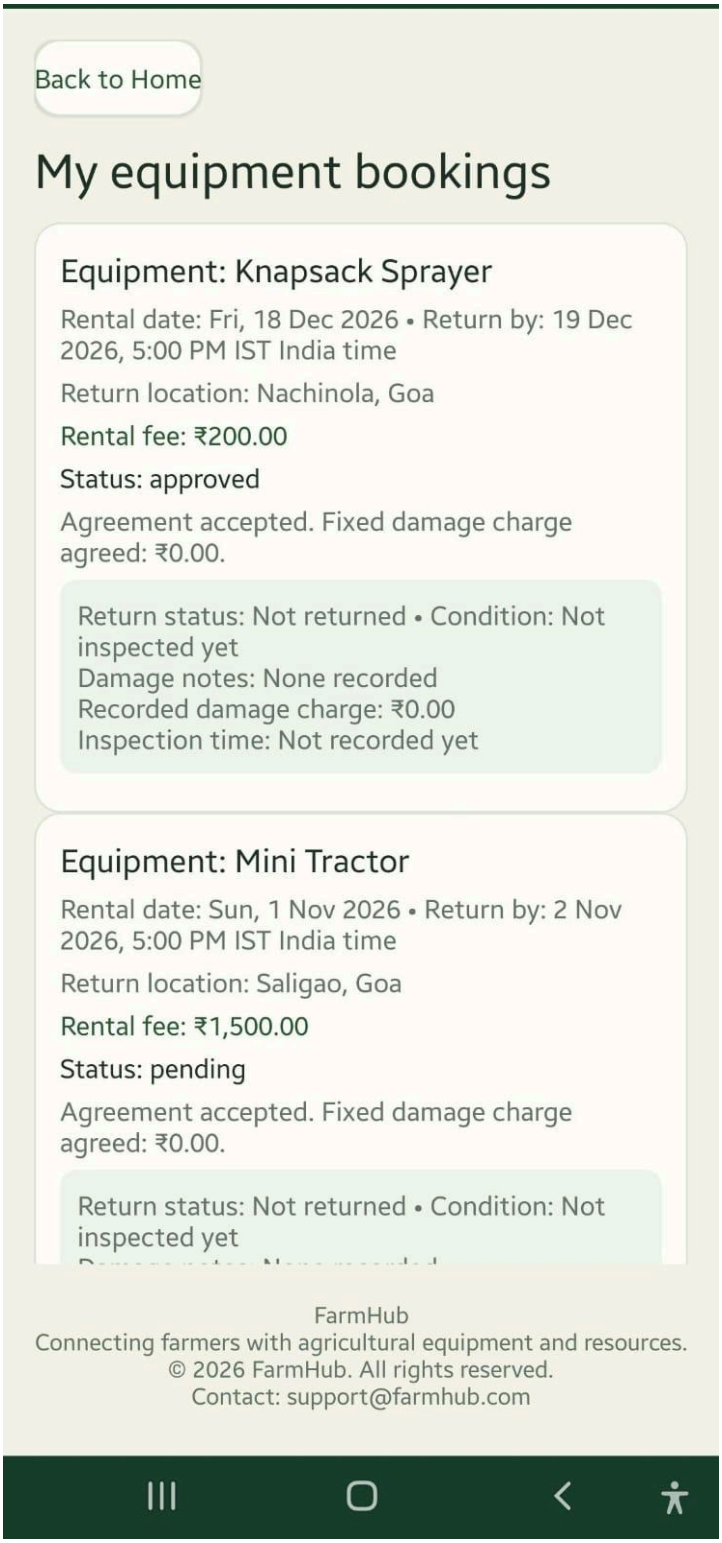
8: Equipment Catalogue and Details

The customer browses available equipment and views its details.



9: Booking and My Bookings

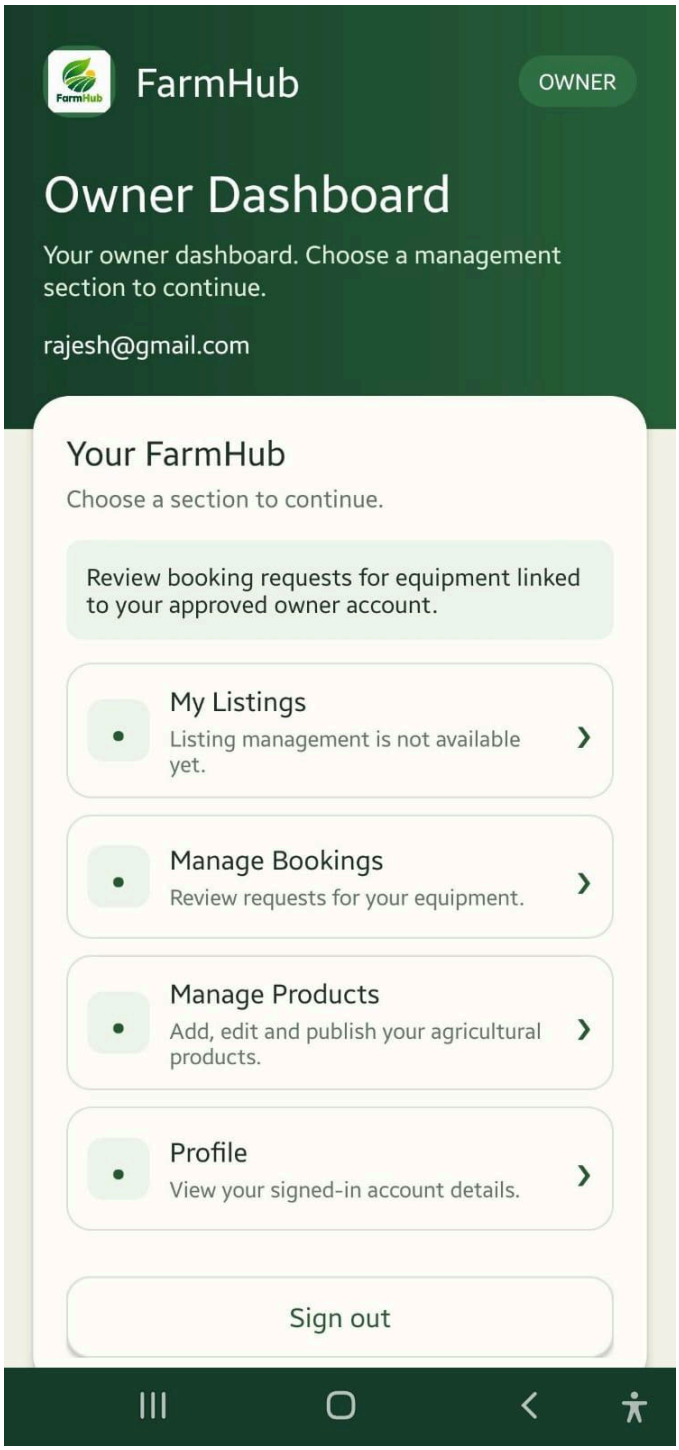
The customer requests equipment rental and views booking information.



Owner Flow

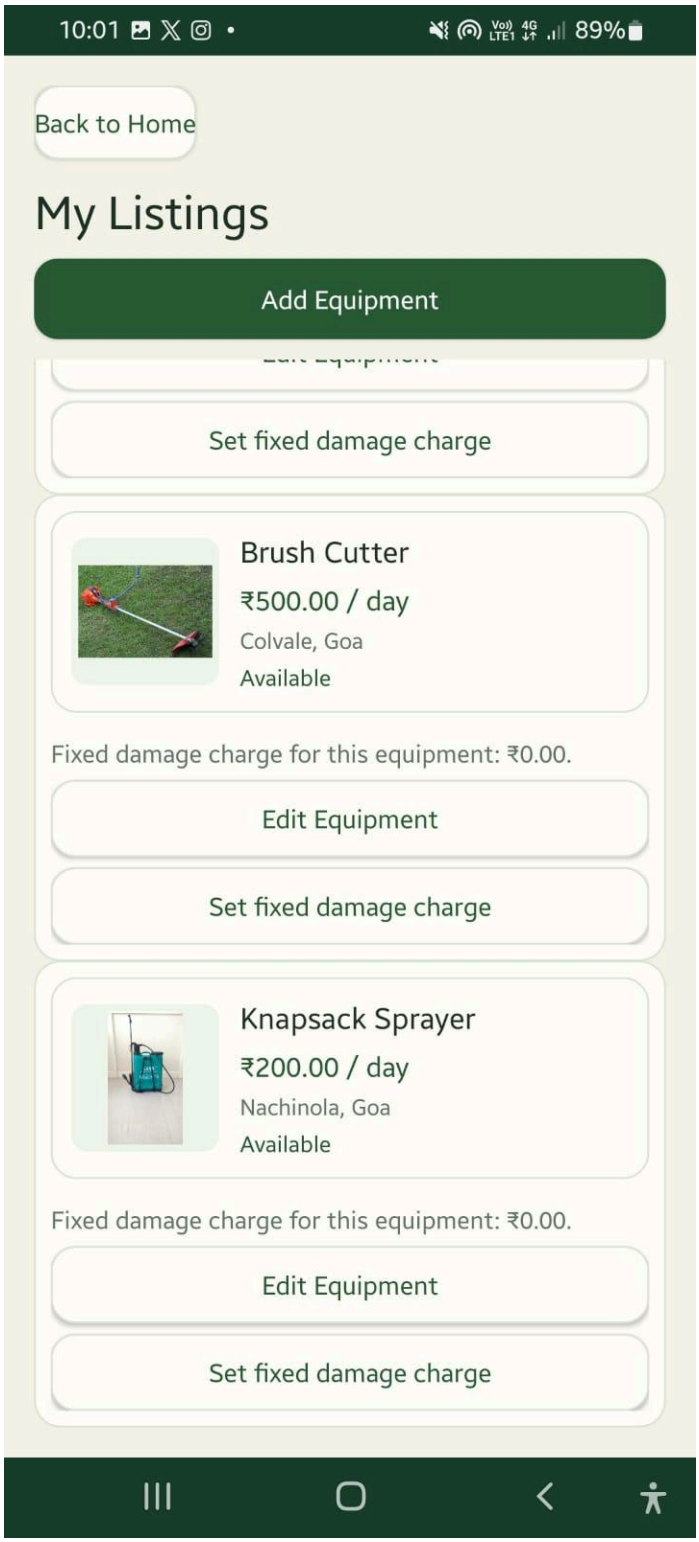
10: Owner Dashboard

The owner signs in and accesses the dashboard to manage FarmHub activities.



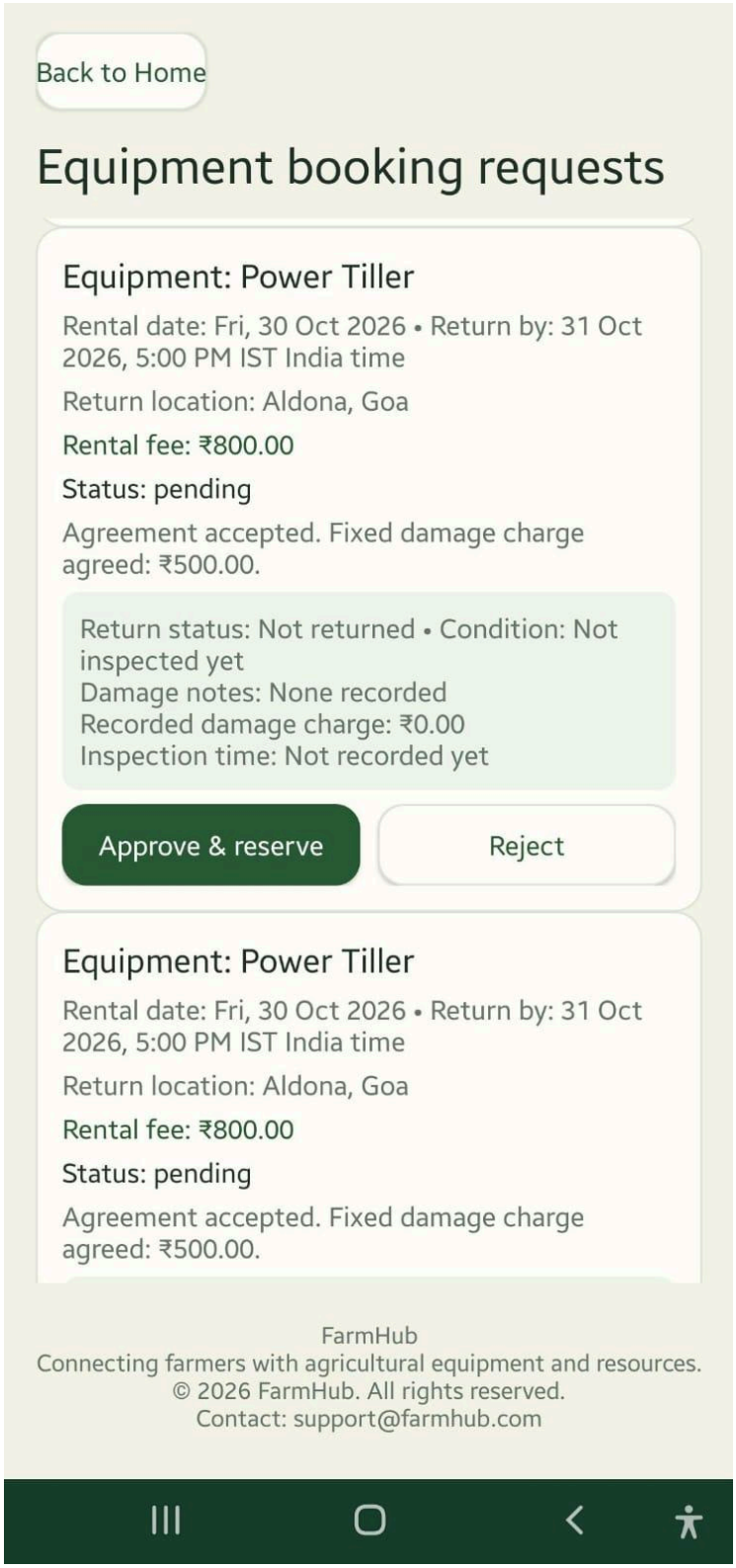
11: My Listings

Allows the owner to view and manage equipment listings. The screenshot indicates that listing management is not available yet.



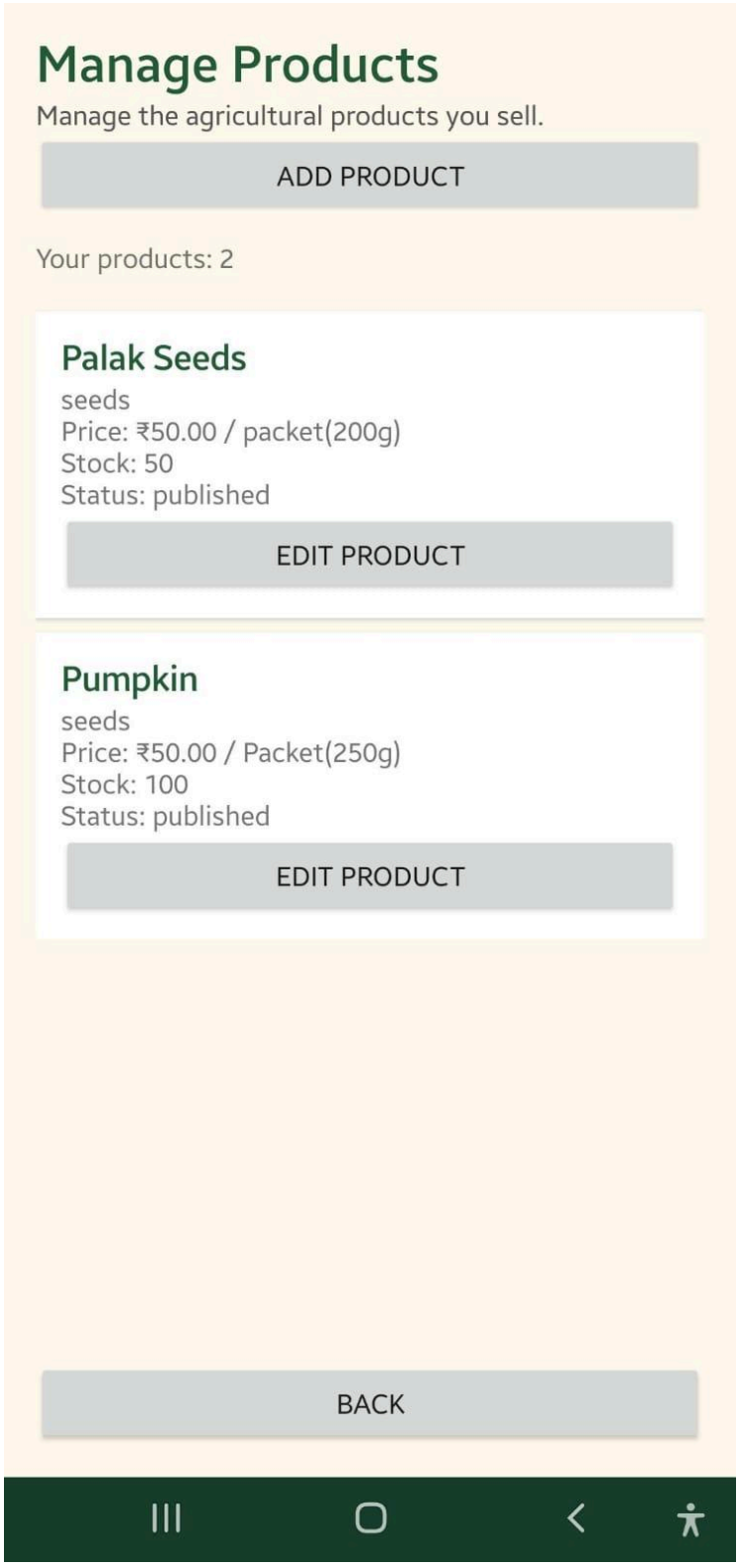
12: Manage Bookings

Allows the owner to review booking requests for their equipment.



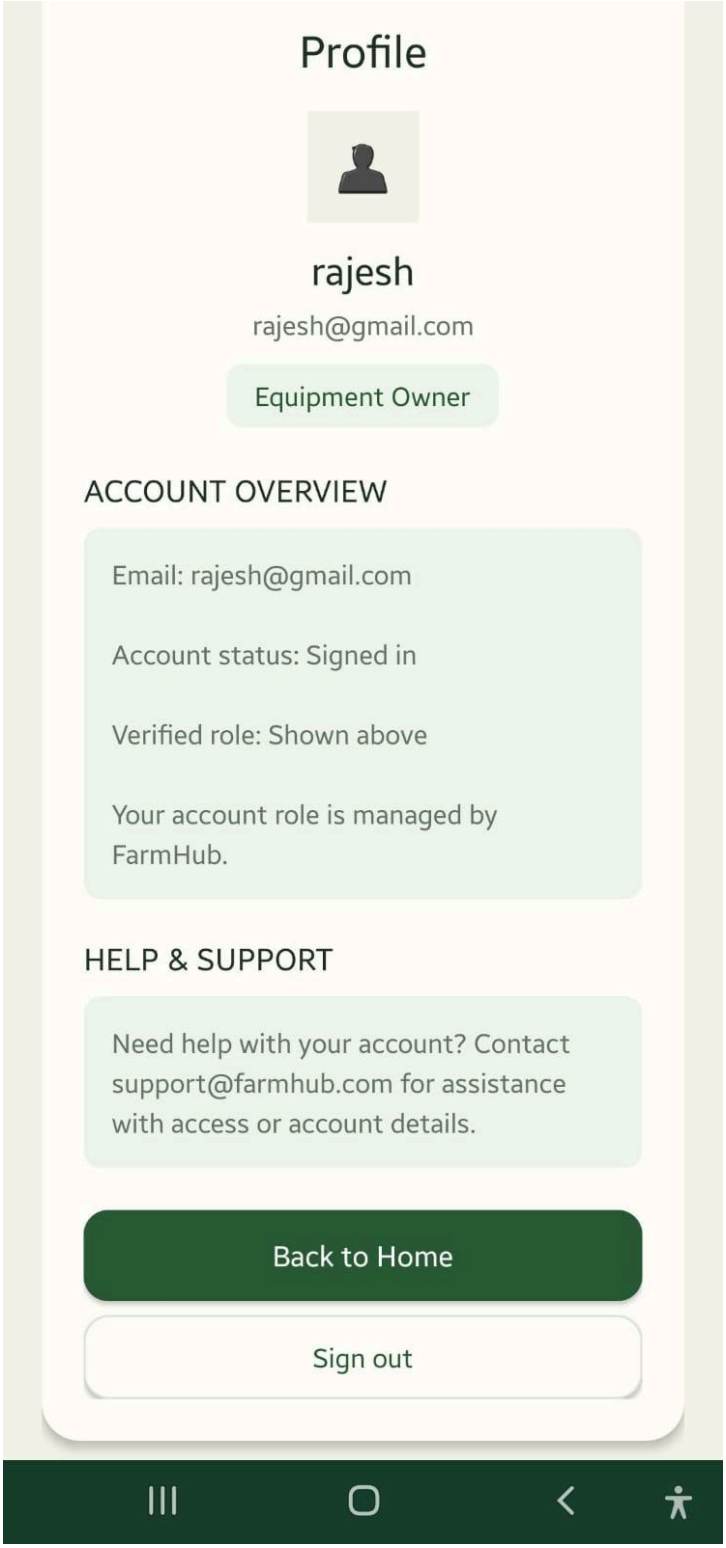
13: Manage Products

Allows the owner to add, edit, and publish agricultural products.



14: Profile

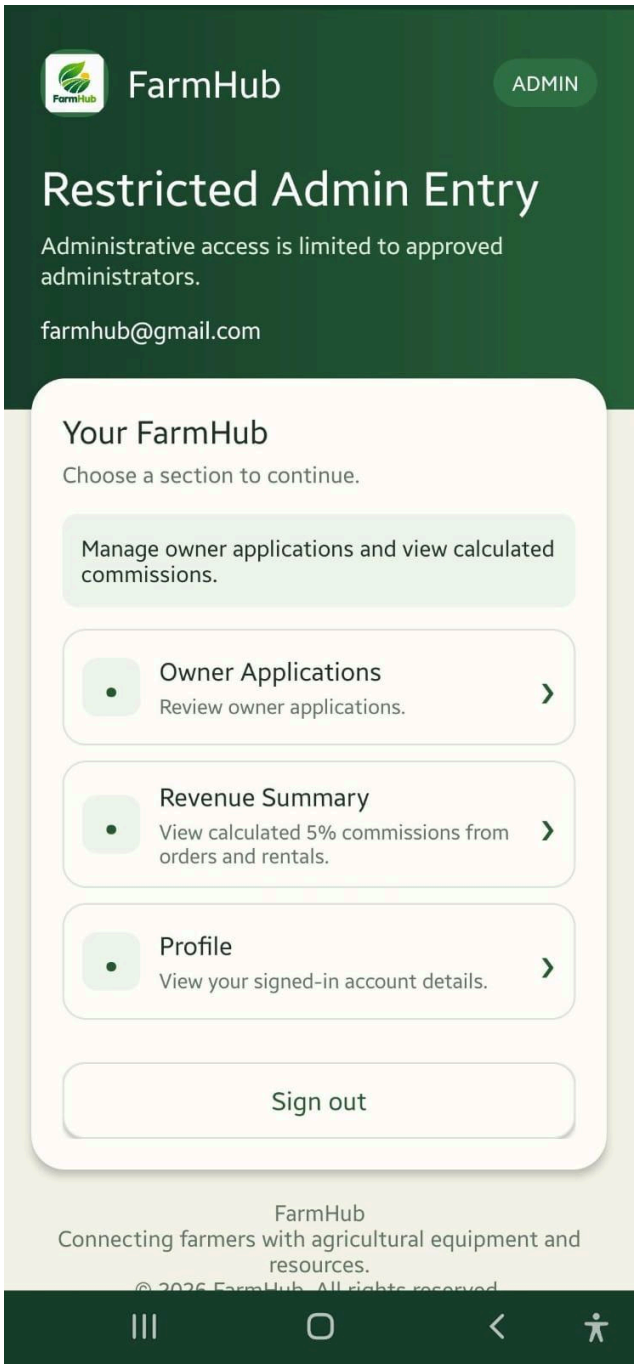
Displays the signed-in owner's account details.



Admin Flow

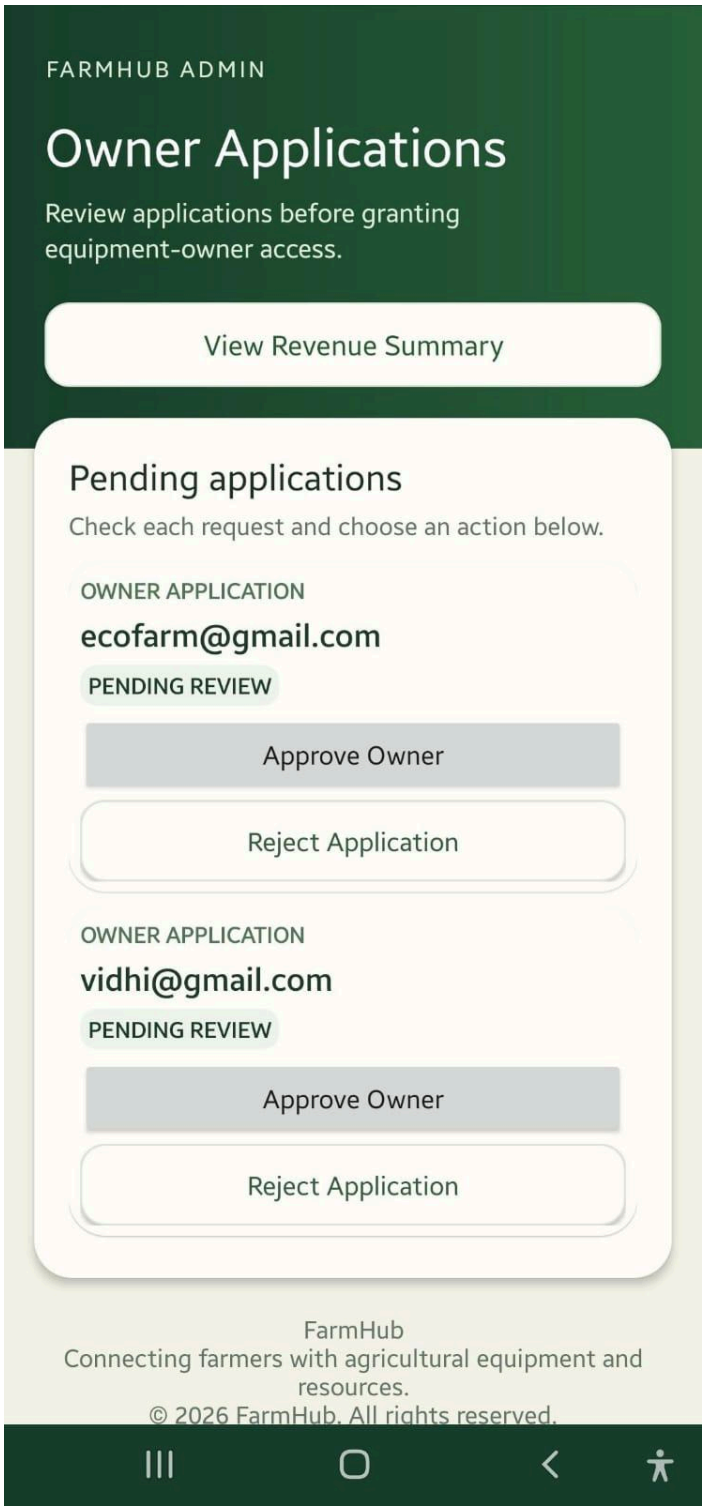
15. Admin Dashboard

The Admin Dashboard provides access to the administrative functions of FarmHub. It allows the Admin to access owner application approval and revenue-related information.



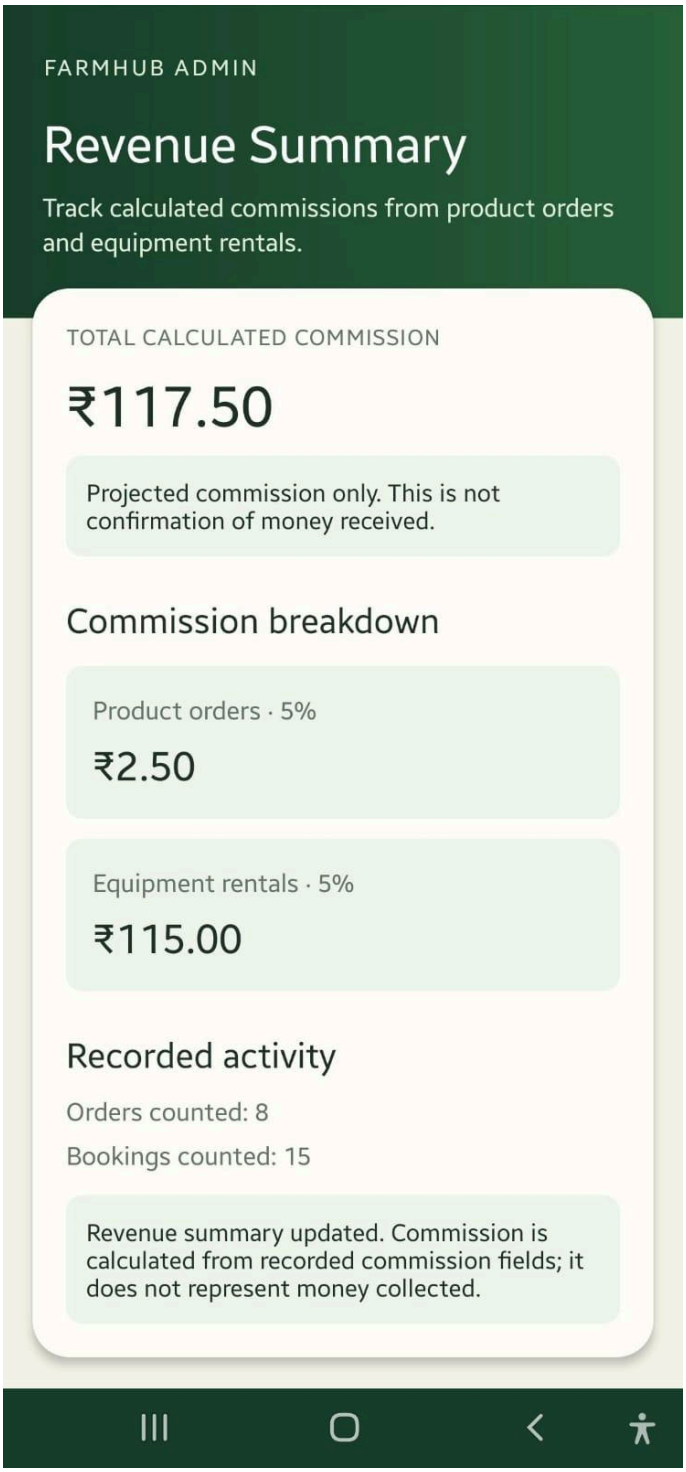
16. Owner Application Approval

The Admin views pending owner applications and can approve or reject applications through the administrative interface.



17. Revenue Summary

The Admin can view the available revenue summary, including product commission, rental commission, total commission, and order and booking counts.



Future Enhancements

1. Online Payment Integration

An online payment gateway can be integrated to allow users to pay securely for agricultural products and equipment rentals using UPI, debit cards, or credit cards.

2. Push Notifications

Notifications can be introduced to inform users about order confirmations, rental booking approvals, booking rejections, and important status updates.

3. Order and Rental Tracking

Users can track product order status, rental booking details, rental duration, and return dates through the application.

4. Improved Admin Dashboard

The Admin Dashboard can be enhanced with detailed reports on users, owners, product orders, equipment bookings, and commission earnings.

5. Product Ratings and Reviews

Users can rate products and share feedback about their shopping and rental experiences, helping other farmers make informed decisions.

6. Enhanced Security

Security can be improved through role-based access control, input validation, and properly configured Firebase security rules to protect user information and application data.

7. Location-Based Services

Location-based features can help users find nearby agricultural products and rental equipment, making it easier to connect with local owners.

References

1. Android Developers, “Get Started with Android Development.” https://developer.android.com/get-started/overview

2. Google Firebase, “Get Started with Firebase Authentication on Android.” Available at: https://firebase.google.com/docs/auth/android/start

3. Google Firebase, “Cloud Firestore Documentation.” https://firebase.google.com/docs/firestore

4. Oracle, “Java Documentation.” Available at: https://docs.oracle.com/en/java/index.html

5. Android Developers, “Meet Android Studio.” https://developer.android.com/studio/intro

6. Google Firebase, “Firebase Authentication Documentation.” https://firebase.google.com/docs/auth/